Rapport de conception – World Of Fun

Plateforme de Jeux Interactifs Web

Présenté par :

- **Saadeddine DOUGHANE**
- **Acile EL DADA**

Formation : Licence 3 MIASHS
ECUE : Applications Web - Semestre 6
Année universitaire : 2025-2026

Lien du site en ligne : https://worldoffun.vercel.app/index.html
Lien du repository GitHub : https://github.com/saaddoughane/World-Of-Fun

Organisation du Repository et des Fichiers

Séparation par jeu
Chaque jeu dispose de son propre dossier contenant tous ses fichiers (HTML, CSS, JS) et ses ressources organisées dans des sous-dossiers dédiés (sounds/, images/, assets/). On a choisi cette organisation de manière intuitive et simple, à l'image des anciens projets que nous avons déjà effectués.

Chaque jeu est isolé des autres, ce qui facilite la maintenance et permet de travailler sur un jeu sans toucher aux autres et donc se séparer le travail de manière plus efficace.

Ce qui veut dire que :

- Le dossier memoire/ contient tous les fichiers pour Mémoire Arctique
- Le dossier chasse-banane/ contient tous les fichiers pour Chasse Bananes
- Le dossier sandy-snake/ contient tous les fichiers pour Snake

Les avantages de cette approche sont nombreux :

- On peut développer un jeu sans affecter les autres
- Facile de tester chaque jeu indépendamment
- Si on veut ajouter un quatrième jeu, il suffit de créer un nouveau dossier
- Chacun de nous peut travailler sur son jeu sans conflit

Séparation des ressources
Dans chaque dossier de jeu, on a séparé les fichiers selon leur nature :

- sounds/ pour les fichiers audio (musique, effets sonores)
- images/ ou assets/ pour les fichiers visuels
- js/ pour les modules JavaScript (notamment pour Chasse Bananes avec son architecture POO)

Cette séparation vient du principe de séparation des responsabilités appris en cours. Le code (logique) est séparé des ressources (médias). Cela facilite :

- La recherche d'un fichier spécifique
- Le remplacement d'une image sans toucher au code
- L'organisation lors du développement en binôme

Assets globaux
Le dossier frontend/assets/ contient les ressources partagées entre plusieurs pages (logo du site, globe terrestre, images de prévisualisation pour le README).

Choix Techniques et Architecture

Une de nos premières décisions de conception était de déterminer quelle technologie utiliser pour chaque jeu, tout en respectant les consignes du projet d'utilisation de DOM et de Canvas.

En cours, nous avons appris que :

- Le DOM est adapté pour des éléments statiques qu'on anime ponctuellement
- Le Canvas est adapté pour des animations continues avec beaucoup d'objets en mouvement

Mémoire Arctique : utilisation du DOM

Les cartes sont créées une seule fois au début du niveau avec createElement(). Elles ne bougent que quand le joueur clique dessus (retournement). Les animations sont gérées en CSS (transitions, transform: rotateY()). Le DOM est parfaitement adapté car les interactions sont ponctuelles.

Chasse Bananes : utilisation de Canvas

Les fruits tombent en permanence et doivent être redessinés 60 fois par seconde. De plus, il y a de nombreux objets en mouvement simultané (fruits + particules de confettis). Le Canvas s'impose pour gérer ce rendu continu et ces animations multiples.

Charming Dunes (Snake) : utilisation de Canvas

Le serpent se déplace continuellement et le jeu nécessite un rendu en temps réel. Le Canvas permet de gérer efficacement les collisions et les mouvements à chaque frame.

Architecture orientée objet (Chasse Bananes)

Pour Chasse Bananes, nous avons adopté une architecture orientée objet car nous avions 4 types de fruits avec des propriétés différentes :

- Banane jaune : lente, 50 points
- Banane verte : moyenne, 100 points
- Régime de bananes : rapide, 200 points
- Ananas : très rapide, 500 points

Approche procédurale (ce qu'on aurait pu faire) :

Créer manuellement chaque fruit avec ses propriétés, puis dupliquer le code de dessin et de mise à jour pour chaque type. Cette approche aurait généré beaucoup de code répétitif et difficile à maintenir.

Approche POO (notre solution) :

Création d'une classe Banana qui sert de modèle. Chaque fruit est une instance de cette classe. La méthode setPropertiesByType() configure automatiquement les propriétés (vitesse, points, taille, image) selon le type passé au constructeur.

Avantages :

- Code réutilisable : créer un fruit = une ligne de code
- Maintenable : modifier les propriétés d'un type = un seul endroit dans le code
- Extensible : ajouter un cinquième fruit = ajouter un cas dans le switch

Cette conception reflète les principes de POO appris en cours : encapsulation (données + méthodes ensemble) et réutilisabilité (classe comme template).

Architecture modulaire (ES6)

Pour Chasse Bananes, nous avons appliqué le principe de séparation des responsabilités en divisant le code en modules :

- banana.js : export de la classe Banana
- particle.js : export de la classe Particle (système de confettis)
- handler.js : export de la gestion des inputs (clavier, souris)
- script.js : import des modules + game loop principal

Cette séparation permet :

- Une meilleure lisibilité (on sait où chercher)
- Un débogage facilité (bug sur les fruits ? → banana.js)
- Une réutilisabilité des modules

Utilisation des Assistants IA

Nous avons utilisé Claude AI et ChatGPT pendant le développement de ce projet.

Rôle de l'IA :

- Aide au débogage d'erreurs spécifiques
- Explication de concepts techniques
- Optimisation de certaines parties du code
- Génération d'éléments visuels (mascottes, icônes)
- Conseils sur les bonnes pratiques JavaScript

L'IA a été utilisée comme un outil d'apprentissage, toute la conception du projet (choix des jeux, architecture des fichiers, organisation par dossier, choix techniques DOM vs Canvas, architecture POO) vient directement de nous et des principes appris en cours.

Répartition des tâches (50/50)

Dès le début du projet, nous nous sommes réparti le travail de façon équilibrée :

Acile EL DADA – 50% :

- Mémoire Arctique (développement complet)
- Chasse Bananes (développement complet)
- Favicons spécifiques à chaque jeu
- Amélioration des éléments visuels de la page d'accueil

Saadeddine DOUGHANE - 50% :

- Page d'accueil avec globe interactif
- Charming Dunes / Snake (développement complet)
- Système d'authentification (inscription, connexion, compte)
- Dashboard des scores
- Déploiement sur Vercel

Collaboration avec Git et GitHub

Nous avons utilisé Git et GitHub pour gérer le versionning et la collaboration, avec des pushs et commits explicites et gestion des conflits quand nécessaire.

Conclusion

Ce projet nous rend vraiment fiers pour plusieurs raisons.

D'abord, on a réussi à créer quelque chose d'original avec le concept du tour du monde. Esthétiquement, on a mis beaucoup d'effort dans la personnalisation : chaque jeu a son univers visuel complet (couleurs, mascottes, curseurs personnalisés), ses propres sons et musiques, et ses animations. On voulait que ce soit beau et immersif, pas juste fonctionnel.

Techniquement, ce projet nous a fait découvrir beaucoup de choses nouvelles. C'était la première fois qu'on utilisait le Canvas 2D pour faire des animations en temps réel, qu'on manipulait le DOM de manière aussi poussée avec les animations 3D, et qu'on structurait un projet de cette ampleur. On a aussi appris à travailler avec Git en équipe, à déployer un site sur Vercel, et à gérer un projet complet de A à Z.

Ce qui nous marque le plus, c'est que c'est notre premier vrai projet web déployé et accessible en ligne. Voir nos jeux fonctionner sur internet, pouvoir les partager avec d'autres personnes, c'est une satisfaction qu'on n'avait jamais eue avant.